

研究生综合应用报告



课程名称_____图论与网络最优化方法_____

姓名	学院	年级	专业	学号	点名册序号
*****	***** 学院	研一	*****	*****	
*****	***** 学院	研一	*****	*****	
*****	***** 学院	研一	*****	*****	

开 课 时 间 2025 至 2026 学 年 第 1 学 期

旅游行程安排优化设计

摘要：本文主要内容为最优哈密尔顿圈搜索问题，即从重庆出发前往道教四大名山，并最终返程重庆，求取其最省时且经济的路线。本文将该问题建模为求取带权无向完全图的 TSP 路径问题，通过将时间成本和经济成本视作权值，建立图，将问题转化为 TSP 问题。通过 TSP 近似算法，即 2-opt 算法，得出近似解。然后再用穷举法求取全局最优解，并进行比较分析，最终得出结论。

关键词：TSP；穷举法；2-opt 方法

1 问题描述

旅游作为文化传播与精神体验的重要载体，宗教文化旅游更是凭借其独特的历史底蕴与人文价值，成为当下备受青睐的出行选择。其中，道教四大名山（青城山、武当山、龙虎山、齐云山）作为我国道教文化的核心圣地，不仅承载着深厚的宗教内涵与历史遗产，其地理位置分布于四川、湖北、江西、安徽四省（跨区域跨度较大），自然风光与人文景观交相辉映，每年吸引大量游客前往朝圣观光。然而，跨区域旅游出行中，人们往往将时间成本（交通耗时）与经济成本（出行费用）视为核心考量因素，二者常存在一定矛盾（如快速出行可能伴随更高费用，低成本路线可能耗时更长）。因此，针对“重庆出发→道教四大名山→返回重庆”的闭环旅游路线，进行科学合理的规划，能够有效平衡两大核心成本，在提升出行效率的同时降低经济负担，为游客提供更优的决策参考。

各个地点的分布图如图 1 所示。



图 1 各地点分布示意图

2 数学建模

2.1 模型假设

基于旅游路线规划的实际场景，为简化模型（时间 - 费用平衡），提出以下假设：

2.1.1 交通约束假设：

（1）出行方式仅考虑“高铁+景区接驳交通”（高铁站→景区的大巴/打车），不考虑自驾、飞机等其他方式，且高铁运行时间、票价及接驳交通的时间/费用均为固定值（不考虑高铁晚点、临时停运、票价浮动等随机因素）；

（2）若两节点间无直达高铁，采用“最短中转路径”（中转次数 ≤ 1 次，优先选择中转候车时间最短的路线），且中转过程无时间损耗（仅计算高铁运行时间+固定候车时间，候车时间按 30 分钟计，符合高铁站常规中转需求）。

2.1.2 节点可达性假设：

（1）5 个核心节点（重庆+道教四大名山对应的高铁站）间均为完全连通（无不可达情况），即任意两节点间均可通过“直达/中转高铁+接驳”实现通行，确保图论模型为完全图 K_5 。

2.1.3 旅游场景假设：

（1）不考虑景区游玩时间、住宿时间等非交通成本，仅聚焦“交通环节的时间-费用优化”；

（2）路线为严格闭环（出发地=返回地=重庆），且每个节点仅访问 1 次（无重复游览，符合 TSP 问题的“访问所有节点一次且仅一次”约束）。

2.2 图论模型定义

2.2.1 节点定义：

将实际旅游中的“核心交通枢纽”抽象为图的节点，共 5 个节点，具体对应关系如下（基于“离景区最近且高铁班次密集”原则选择高铁站，以下费用以及时间数据来自高德地图。）：

v_1 ：重庆西站（出发/返回地，对应重庆）；

v_2 ：都江堰站（对应青城山，景区接驳：都江堰站→青城山景区大巴，耗时 50 分钟，费用 17 元）；

v_3 ：十堰东站（对应武当山，景区接驳：十堰东站→武当山景区大巴，耗时 70 分钟，费用 14 元）；

v_4 : 鹰潭北站 (对应龙虎山, 景区接驳: 鹰潭北站→龙虎山景区大巴, 耗时 60 分钟, 费用 20 元) ;

v_5 : 黄山北站 (对应齐云山, 景区接驳: 黄山北站→齐云山景区大巴, 耗时 70 分钟, 费用 2 元) 。

节点集记为: $V = \{v_1, v_2, v_3, v_4, v_5\}$

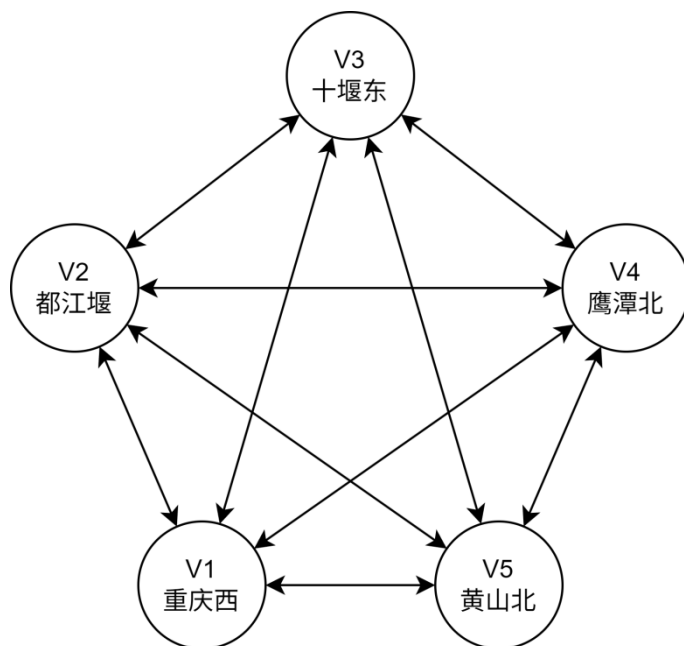


图 2 数学建模示意图

2.2.2 边与权值定义:

将“两节点间的交通成本”抽象为图的边与权值, 每条边 $(v_i, v_j) \in E (i \neq j)$ 对应“从节点 v_i 到 v_j 的单程交通”, 定义两类权值:

(1) 时间权值 w_{ij}^t

从 v_i 到 v_j 的总交通耗时 (单位: 小时), 含“高铁运行时间+中转候车时间 (若有)+景区接驳时间”;

(2) 费用权值 w_{ij}^c

从 v_i 到 v_j 的总交通费用 (单位: 元), 含“高铁票价 (直达/中转合计)+景区接驳费用”。边集记为 $E = \{(v_i, v_j) | i, j \in \{1, 2, 3, 4, 5\}, i \neq j\}$, 权值矩阵分别记为 $\mathbf{W}^t = [w_{ij}^t]_{5 \times 5}$ (时间权值矩阵)、 $\mathbf{W}^c = [w_{ij}^c]_{5 \times 5}$ (费用权值矩阵)。

2.2.3 TSP 问题对应

实际问题的“重庆出发→玩遍四大名山→返回重庆”，对应图论中“寻找一条经过所有节点 $v_1 \sim v_5$ 一次且仅一次、最后回到起点 v_1 的闭环路径”，目标是“最小化加权后的总权值”（后续求解章节的核心目标），即标准的“对称 TSP 问题”（因 $w_{ij}^t = w_{ji}^t$ 、 $w_{ij}^c = w_{ji}^c$ ，往返交通成本一致）。

2.3 数据来源与预处理

2.3.1 数据来源

- （1）高铁运行时间、票价：2025 年 12 月通过“12306 官网”查询（选择每日固定班次的二等座票价，避免高峰时段浮动票价）；
- （2）景区接驳时间、费用：2025 年 12 月通过“高德地图”的“公交/打车规划”功能查询（取常规时段的平均耗时与费用）；
- （3）中转路径：若两节点无直达高铁（如 $v_1 \leftrightarrow v_5$ ），通过 12306“中转联程”功能选择“最短耗时+最低费用”的中转方案（如 $v_1 \rightarrow$ 武汉站 $\rightarrow v_5$ ）。

2.3.2 原始数据表格

基于上述来源，整理 5 节点间的时间权值 w_{ij}^t （单位：小时）与费用权值 w_{ij}^c （单位：元），结果如下表所示（注：表格中“—”表示无直达高铁，“（中转）”表示需中转，数据为中转后合计值）：

表 1 节点间时间权值矩阵（单位：小时）

从 v_i 到 v_j	v_1 (重庆西)	v_2 (都江堰)	v_3 (十堰东)	v_4 (鹰潭北)	v_5 (黄山北)
v_1 (重庆西)	—	6.1	4.5（中转）	8.1	9.2（中转）
v_2 (都江堰)	6.1	—	15.2（中转）	12（中转）	14（中转）
v_3 (十堰东)	4.5（中转）	15.2（中转）	—	8.2（中转）	7.1（中转）
v_4 (鹰潭北)	8.1	12（中转）	8.2（中转）	—	1.5
v_5 (黄山北)	9.2（中转）	14（中转）	7.1（中转）	1.5	—

表 2 节点间费用权值矩阵（单位：元）

从 v_i 到 v_j	v_1 (重庆西)	v_2 (都江堰)	v_3 (十堰东)	v_4 (鹰潭北)	v_5 (黄山北)
v_1 (重庆西)	—	163	421（中转）	679	792
v_2 (都江堰)	163	—	138（中转）	841（中转）	978.5（中转）
v_3 (十堰东)	421（中转）	138（中转）	—	720（中转）	516.5（中转）
v_4 (鹰潭北)	679	841（中转）	720（中转）	—	127
v_5 (黄山北)	792	978.5（中转）	516.5（中转）	127	—

2.3.3 数据预处理说明

后续双目标加权优化需消除量纲差异（时间：小时/费用：元），此处暂不展开标准化计算，仅确保原始数据的准确性与完整性（如中转路径的时间/费用已合并计算，接驳成本已纳入总权值）。

3 双目标加权 TSP 模型的 2-opt 算法求解

3.1 双目标转化为单目标（时间成本经济化）

在第二章构建的时间权值（小时）与费用权值（元）矩阵中，由于量纲不统一，无法直接加权优化。相较于传统 min-max 标准化，采用“时间成本经济化”的方式更贴合旅游出行的实际决策逻辑——游客的时间成本可等价为“放弃工作的机会成本”，因此以重庆本地平均时薪作为转化系数，将时间成本直接转化为经济成本，实现双目标向单目标的统一。

3.1.1 转化逻辑与公式

参考重庆市 2025 年城镇非私营单位就业人员平均时薪（约 30 元/小时），定义“总经济成本”为“直接交通费用+时间机会成本”，具体公式如下：

对任意边 $(v_i, v_j) \in E$ ，其单目标权值（总经济成本）为：

$$w_{ij}=w_{ij}^c+30\times w_{ij}^t$$

其中：

w_{ij}^c ：第二章定义的边 $((v_i, v_j))$ 原始费用权值（元，含高铁 + 接驳费用）；

w_{ij}^t ：第二章定义的边 $((v_i, v_j))$ 原始时间权值（小时，含高铁 + 接驳时间）；

30：重庆平均时薪（元 / 小时），体现 “1 小时时间成本等价于 30 元经济成本”。

3.1.2 单目标权值矩阵计算

通过上述公式计算出单位节点间单目标总经济成本矩阵如下。

表 3 节点间单目标总经济成本矩阵（单位：元）

从 v_i 到 v_j	v_1 (重庆西)	v_2 (都江堰)	v_3 (十堰东)	v_4 (鹰潭北)	v_5 (黄山北)
v_1 (重庆西)	—	346	556	922	1068
v_2 (都江堰)	346	—	594	1207	1398.5
v_3 (十堰东)	556	594	—	318	729.5
v_4 (鹰潭北)	922	1207	318	—	172
v_5 (黄山北)	1068	1398.5	729.5	172	—

3.2 2-opt 算法原理

2-opt 算法是解决 TSP 问题的经典局部搜索优化算法，核心思想是通过“消除路径中的交叉边”迭代优化初始路径，最终得到近似最优解。该算法具有实现简单、收敛速度快、对小规模 TSP（如本问题 $n=5$ ）优化效果好的特点，适配本研究的场景需求。

3.2.1 算法核心思想

对于 TSP 的闭环路径 $P = [v_1 \rightarrow v_{i_1} \rightarrow v_{i_2} \rightarrow \cdots \rightarrow v_{i_{n-1}} \rightarrow v_1]$ ，2-opt 算法通过以下步骤优化：

1. 随机/贪心生成一条初始闭环路径；
2. 遍历路径中所有不相邻的边对 $(v_a \rightarrow v_{a+1}, v_b \rightarrow v_{b+1}) (b > a + 1)$ ；
3. 移除该边对，替换为 $(v_a \rightarrow v_b, v_{a+1} \rightarrow v_{b+1})$ ，形成新路径；
4. 计算新路径的总权值，若小于原路径则更新路径；
5. 重复步骤 2-4，直至遍历所有边对后无更优路径，输出当前最优路径。

3.2.2 算法优势

首先本问题仅 5 个节点，2-opt 算法可在极短时间内完成迭代，且能保证近似解接近理论最优解；其次，本方法无需复杂数学推导，可通过 Python 快速编码实现，贴合课程报告“算法落地”的要求；此外，相较于贪心算法等简单启发式算法，2-opt 能有效降低路径总权值，提升解的质量。

3.3 算法设计与实现

本节基于 Python 语言实现 2-opt 算法，以表 3 的单目标权值矩阵为输入，生成初始路径并迭代优化，最终输出最优闭环路径及总经济成本。

代码请见附录，运行结果如下：2-opt 优化后路径：['重庆西','都江堰','十堰东','鹰潭北','黄山北','重庆西']；优化后总经济成本：2498.0 元。

4 结果评估

4.1 穷举法求取全局最优解

代码实现请见附录，运行结果如下：

=== 全局最优路径（穷举法） ===

最优路径：重庆西→都江堰→十堰东→鹰潭北→黄山北→重庆西

总经济成本：2498.0 元

总耗时（高铁+接驳）：40.199999999999996 小时

总费用（高铁+接驳）：1940.0 元

4.2 结果比对

两类算法求解结果核心对比如下表所示：

表 4 不同方法求解 TSP 问题结果对比

对比维度	2-opt 算法求解结果	穷举法求解结果	核心结论
最优闭环	重庆西→都江堰→十堰东→鹰	重庆西→都江堰→十堰东→鹰	路径完全
路径	潭北→黄山北→重庆西	潭北→黄山北→重庆西	一致
总经济成本	2498.0 元	2498.0 元	成本完全相等
总耗时/ 总费用	40.2 小时 / 1940.0 元	40.2 小时/1940.0 元	核心指标完全匹配
算法效率	瞬间完成求解	需枚举 12 条路径, 计算量略大	2-opt 算法更高效

4.3 分析总结

- （1）建模有效性：本研究将旅游闭环路线规划转化为带权无向完全图的 TSP 问题，整合时间与费用双成本构建权值矩阵，通过“时间成本经济化”实现双目标统一，模型贴合跨区域旅游的实际需求。
- （2）算法适配性：2-opt 算法在节点数较少的场景下，能快速收敛至全局最优解，无需复杂计算；穷举法虽能保证结果绝对最优，但计算量随节点数指数增长，

仅适用于小规模问题验证。

（3）实际价值：最优路径“重庆西→都江堰（青城山）→十堰东（武当山）→鹰潭北（龙虎山）→黄山北（齐云山）→重庆西”，实现了时间与经济成本的均衡，为游客提供了直接可参考的行程方案。

附录 1：2-opt 算法求取 TSP 问题结果

```
import numpy as np
```

```
# 1. 定义节点与权值矩阵（对应表 3-1，v1=0, v2=1, v3=2, v4=3, v5=4）
```

```
nodes = ["重庆西", "都江堰", "十堰东", "鹰潭北", "黄山北"]
```

```
# 权值矩阵（行：出发节点，列：到达节点，-1 表示自身无连接）
```

```
w_matrix = np.array(  
    [  
        [-1, 346, 556, 922, 1068],  
        [346, -1, 594, 1207, 1398.5],  
        [556, 594, -1, 318, 729.5],  
        [922, 1207, 318, -1, 172],  
        [1068, 1398.5, 729.5, 172, -1],  
    ]  
)
```

```
# 2. 计算路径总权值的函数
```

```
def calculate_total_cost(path, w_matrix):  
    total = 0  
    n = len(path)  
    for i in range(n - 1):  
        from_node = path[i]  
        to_node = path[i + 1]  
        total += w_matrix[from_node, to_node]  
    return total
```

```
# 3. 2-opt 算法核心函数
```

```

def tsp_2opt(initial_path, w_matrix):
    path = initial_path.copy()
    n = len(path)
    improved = True
    while improved:
        improved = False
        # 遍历所有可能的边对(a, b)
        for a in range(1, n - 2):
            for b in range(a + 1, n - 1):
                # 生成新路径：反转 a 到 b 的节点
                new_path = path[:a] + path[a : b + 1][::-1] + path[b + 1 :]
                # 计算新旧路径的总权值
                old_cost = calculate_total_cost(path, w_matrix)
                new_cost = calculate_total_cost(new_path, w_matrix)
                # 若新路径更优，则更新
                if new_cost < old_cost:
                    path = new_path
                    improved = True
    # 计算最终总权值
    total_cost = calculate_total_cost(path, w_matrix)
    return path, total_cost

```

4. 生成初始路径（贪心算法：从重庆西(0)出发，每次选当前节点的最小权值未访问节点）

```

def greedy_initial_path(start_node, w_matrix):
    n = w_matrix.shape[0]
    visited = [False] * n
    path = [start_node]
    visited[start_node] = True

```

```

current = start_node
# 遍历剩余 n-1 个节点
for _ in range(n - 1):
    min_cost = float("inf")
    next_node = -1
    # 找当前节点到未访问节点的最小权值
    for i in range(n):
        if (
            not visited[i]
            and w_matrix[current, i] != -1
            and w_matrix[current, i] < min_cost
        ):
            min_cost = w_matrix[current, i]
            next_node = i
    path.append(next_node)
    visited[next_node] = True
    current = next_node
# 闭环：回到起点
path.append(start_node)
return path

```

5. 运行算法

```

if __name__ == "__main__":
    # 生成贪心初始路径（起点为重庆西，对应索引 0）
    initial_path = greedy_initial_path(0, w_matrix)
    print("贪心初始路径: ", [nodes[i] for i in initial_path])
    print("初始路径总经济成本: ", calculate_total_cost(initial_path, w_matrix), "元")

```

2-opt 优化

```
opt_path, opt_cost = tsp_2opt(initial_path, w_matrix)
print("\n2-opt 优化后路径: ", [nodes[i] for i in opt_path])
print("优化后总经济成本: ", opt_cost, "元")
```

附录 2：穷举法求取 TSP 问题全局最优解

```
import numpy as np

from itertools import permutations

# 1. 定义节点与权值矩阵（与第三章一致）
nodes = ["重庆西", "都江堰", "十堰东", "鹰潭北", "黄山北"]
node_index = {0: "重庆西", 1: "都江堰", 2: "十堰东", 3: "鹰潭北", 4: "黄山北"}

# 单目标总经济成本矩阵（元）
w_matrix = np.array(
    [
        [-1, 346, 556, 922, 1068],
        [346, -1, 594, 1207, 1398.5],
        [556, 594, -1, 318, 729.5],
        [922, 1207, 318, -1, 172],
        [1068, 1398.5, 729.5, 172, -1],
    ]
)

# 原始时间权值矩阵（小时，用于后续还原时间维度）
t_matrix = np.array(
    [
        [-1, 6.1, 4.5, 8.1, 9.2],
        [6.1, -1, 15.2, 12, 14],
        [4.5, 15.2, -1, 8.2, 7.1],
        [8.1, 12, 8.2, -1, 1.5],
        [9.2, 14, 7.1, 1.5, -1],
    ]
)

# 原始费用权值矩阵（元，用于后续还原费用维度）
c_matrix = np.array(
```

```
[
    [-1, 163, 421, 679, 792],
    [163, -1, 138, 841, 978.5],
    [421, 138, -1, 720, 516.5],
    [679, 841, 720, -1, 127],
    [792, 978.5, 516.5, 127, -1],
]
```

2. 计算路径总权值的函数（支持经济成本/时间/费用维度）

```
def calculate_path_cost(path, matrix):
```

```
    total = 0
```

```
    n = len(path)
```

```
    for i in range(n - 1):
```

```
        from_node = path[i]
```

```
        to_node = path[i + 1]
```

```
        total += matrix[from_node, to_node]
```

```
    return total
```

3. 穷举所有唯一闭环路径

```
def exhaustive_tsp(start_node, w_matrix, t_matrix, c_matrix):
```

```
    # 节点总数
```

```
    n = w_matrix.shape[0]
```

```
    # 剩余节点（排除起点）
```

```
    remaining_nodes = [i for i in range(n) if i != start_node]
```

```
    # 生成剩余节点的全排列
```

```
    permutations_list = permutations(remaining_nodes)
```

```
    # 存储所有唯一路径及对应成本（经济成本、时间、费用）
```

```
    path_info = []
```

```
    # 存储已遍历的路径（避免往返重复）
```

```

visited_paths = set()

for perm in permutations_list:
    # 构建闭环路径：起点→排列节点→起点
    path = [start_node] + list(perm) + [start_node]
    # 生成路径的字符串表示（用于判断往返重复）
    path_str = "→".join([str(node) for node in path[:-1]])
    # 生成反向路径的字符串表示
    reverse_path_str = "→".join([str(node) for node in reversed(path[:-1])])

    # 若反向路径已遍历，跳过
    if reverse_path_str in visited_paths:
        continue
    visited_paths.add(path_str)

    # 计算该路径的总经济成本、总时间、总费用
    total_economic = calculate_path_cost(path, w_matrix)
    total_time = calculate_path_cost(path, t_matrix)
    total_cost = calculate_path_cost(path, c_matrix)

    # 存储路径信息（节点索引、路径名称、三类成本）
    path_name = "→".join([node_index[node] for node in path])
    path_info.append(
        {
            "path_index": path,
            "path_name": path_name,
            "total_economic": total_economic,
            "total_time": total_time,
            "total_cost": total_cost,
        }
    )

```

```

    )

# 找到全局最优路径（总经济成本最小）
optimal_path = min(path_info, key=lambda x: x["total_economic"])
return path_info, optimal_path

# 4. 运行穷举法（起点为重庆西，索引 0）
if __name__ == "__main__":
    all_paths, optimal_path = exhaustive_tsp(
        start_node=0, w_matrix=w_matrix, t_matrix=t_matrix, c_matrix=c_matrix
    )

    # 输出所有路径信息
    print("=== 所有 12 条闭环路径的成本信息 ===")
    for idx, path in enumerate(all_paths, 1):
        print(f'路径 {idx}: {path["path_name"]}')
        print(
            f' 总经济成本: {path["total_economic"]} 元, 总耗时: {path["total_time"]}
            小时, 总费用: {path["total_cost"]} 元\n'
        )

    # 输出全局最优路径
    print("=== 全局最优路径（穷举法） ===")
    print(f'最优路径: {optimal_path["path_name"]}')
    print(f'总经济成本: {optimal_path["total_economic"]} 元')
    print(f'总耗时（高铁+接驳）: {optimal_path["total_time"]} 小时')
    print(f'总费用（高铁+接驳）: {optimal_path["total_cost"]} 元')

```